Queryable AAS Graphs for AI Agents: An Event-Driven Knowledge Graph Integration for AAS Environments

Gerhard Sonnenberg*, Peter Stein+, Fabio Espinosa*, Daniel Porta*
*Cognitive Assistants & +Innovative Factory Systems
German Research Center for Artificial Intelligence (DFKI)
*Saarbrücken & +Kaiserslautern, Germany
{firstname.lastname@dfki.de}

***Abstract*—The Asset Administration Shell (AAS) plays a key role in Industry 4.0, providing a standardized digital representation of industrial assets to enable interoperability and data exchange. However, information retrieval with complex queries is still in its infancy despite recent advances in the specification of a dedicated query language, which will be hard to implement for different SDKs and storage back-ends.**

This paper proposes an event-driven architecture that integrates AAS contents into a Neo4j-based knowledge graph using Apache Kafka. This enables powerful, relationship-oriented Cypher queries for complex information retrieval and supports advanced cross-validation of references. The knowledge graph will be kept in sync with changes in the AAS. This establishes a solid foundation for advanced natural language user interfaces. As a proof-of-concept, we implemented an AI agent using a Large Language Model (LLM) and a standardized Neo4j tool integration by means of the Model Context Protocol (MCP) for question answering.

Thus, the paper contributes to making the AAS more accessible and actionable for AI-driven industrial applications in a broad range of use cases without the need for a dedicated query language. The source code is publicly available on GitHub.

***Index Terms*—Asset Administration Shell, Kafka, Neo4j, Knowledge Graph, LLM, MCP, AI Agent, Digital Twin.**

I. Introduction

The fourth industrial revolution, Industry 4.0, promises a profound transformation of production through the digital networking of assets. Central to this vision is the Asset Administration Shell (AAS) [1], a standardized digital twin representing physical assets to enable interoperability and seamless data exchange across organizational boundaries. The AAS encapsulates all relevant information and functionalities of an asset throughout its life-cycle, serving as a key enabler of digital transformation.

There is however still a clear lack of integrated, semantically aware query capabilities that leverage asset relationships and their technical and semantic properties. Current services and tools such as the AAS Web UI [2] support basic viewing and editing within individual shells but offer only limited query capabilities. These primarily rely on ID-based searches (e.g., `globalAssetId` or simple tags). Achieving an aggregated, semantically rich, and validated view across multiple AAS instances is cumbersome or often infeasible with existing interfaces. This forces users to manually aggregate and process data from various AAS endpoints at the client-side, which is inefficient and error-prone, especially in large-scale industrial environments.

The inability to perform complex cross-shell and cross-repository queries and cross-validate references such as `semanticId` or `globalAssetId` undermines data quality and trust in interconnected digital twins. This hinders advanced data-driven applications, especially across value and supply chains, and highlights the need for a more robust and flexible architecture that can effectively manage, query, and validate the complex, interconnected, and semantically rich data structures inherent in Asset Administration Shells.

The latest AAS API specification (V3.1) [3] published by the Industrial Digital Twin Association (IDTA) introduces a query language that needs to be implemented by registry and repository providers and needs to be mapped to specific persistence back-ends. While for extremely large AAS repositories this can be beneficial in terms of speed and load, it is currently limited to isolated repositories. In other usage scenarios, a centrally managed search index over multiple registries and repositories as presented in this paper is more feasible.

We propose an event-driven architecture that integrates AAS contents into a Neo4j-based knowledge graph using Apache Kafka. This enables powerful, relationship-oriented Cypher queries [4] for complex information retrieval and supports advanced cross-validation of references. The knowledge graph will be kept in sync with changes in the AAS. This establishes a solid foundation for, e.g., advanced natural language user interfaces. As a proof-of-concept, we implemented an AI agent using a Large Language Model (LLM) and a standardized Neo4j tool integration by means of the Model Context Protocol (MCP) for question answering.

After reviewing the state of the art focusing on the integration of the AAS with knowledge graphs in Section II, we present our event-driven approach in Section III. Section IV describes in detail the efficient mapping of the AAS metamodel to Neo4j. Section V discusses the evaluation of our approach. Section VI highlights an agentic use case on top of the knowledge graph. We conclude in Section VII.

II. State of the Art

A. Knowledge Graph Formalisms for AAS Representation

Two prominent formalisms for modeling knowledge graphs are Labelled Property Graphs (LPG) and OWL (Web Ontology Language) ontologies, each with distinct theoretical underpinnings and practical affordances. Both conceptualize interconnected data as graphs comprising nodes (entities) and edges (relationships). They support rich metadata annotation, enabling contextualization of nodes and edges through key-value properties or axiomatic statements. This facilitates advanced querying and reasoning tasks across diverse domains.

The primary distinction lies in their semantic expressivity, formal rigor and compactness. OWL ontologies, grounded in Description Logics, enable logical inference through axioms and class hierarchies, supporting reasoning tasks such as consistency checking and subsumption inference. In contrast, LPGs prioritize flexibility and performance, allowing arbitrary key-value pairs directly on nodes and edges but lacking formal semantics for logical entailment. Consequently, OWL is suited for domains requiring rigorous ontological commitments and automated reasoning with the terminology known in advance, whereas LPGs are favored in scenarios demanding scalability, schema agility, and ease of integration with graph database systems like, e.g., Neo4j.

B. Related Work

Research on making the AAS queryable falls into two broad lines: (i) ontology-driven approaches that translate AAS data into an RDF/OWL knowledge graph and (ii) graph database approaches that store AAS structures natively in a labelled property graph such as Neo4j. A concise overview is given below.

a) Ontology–based knowledge graphs: Grangel-González et al. [5] pioneered the idea of a *Semantic AAS* by publishing an OWL vocabulary, which mirrors the official AAS meta-model. Their offline converter allows AAS instances to be turned into RDF triples and queried with SPARQL, but does not support continuous synchronization. Bozkurt and Schulz [6] extend this line of work to production–logistics planning in fluid manufacturing systems. By integrating product, process and resource (PPR) Submodels into an RDF triple store they enable rule-based SPARQL queries for tasks such as load-carrier selection; the setting, however, is essentially static and batch-oriented. Rimaz [7] offers a comprehensive OWL formalization of the entire AAS information model, including SHACL constraints for data quality checks, again targeting offline data integration.

b) LPG representations in Neo4j: Several recent papers choose Neo4j to exploit its efficient path traversal and Cypher query language. Rübel et al. [8] create a Neo4j graph to support skill-based fault diagnosis and refer to Kafka-style event modeling from previous work, yet their prototype focuses on the static context graph and omits a streaming implementation. Schmeyer et al. [9] propose a domain-specific data broker that maps energy sector AAS artefacts into Neo4j. Semantic alignments between Submodels and the Open Energy Ontology are generated semi-automatically with LLM assistance; the integration workflow is interactive but not intended for real-time pipelines. Luxenburger et al. [10] describe a microservice infrastructure for I4.0 testbeds where the AAS registry emits (un)registration events of shells and Submodels via Apache Kafka. A Neo4j service maintains a live structural mirror that can be queried with Cypher. The work presented in this paper is a major update of that service. It now relies on V3 of the AAS metamodel and additionally considers fine-grained change events emitted by distributed AAS or Submodel repositories.

c) Position of this work: In contrast to ontology-centric solutions, our architecture embraces Neo4j as a *native runtime store* and couples it with *event streaming* through Apache Kafka. Unlike previous Neo4j-based studies, the entire ingestion path from fine-grained CRUD events to Cypher transactions is fully automated and evaluated under realistic update loads. This enables (i) real-time cross-shell queries, (ii) structural cross-validation such as dangling-reference detection and (iii) a simple migration path towards natural-language agents, all *without* a dedicated query language for AAS. The next section details the technical architecture.

III. Event-driven Knowledge Graph Integration for AAS Environments

To address the identified limitations of existing AAS data management and query capabilities, we propose a novel event-driven architecture that leverages the strengths of Apache Kafka for robust data streaming and Neo4j for powerful graph-based data representation and querying. This architecture ensures data synchronization, enhanced structural validation, and flexible query possibilities for AAS instances.

The overall architecture, illustrating the flow from the AAS environment to Neo4j, is shown in Fig. 1 and in detail explained in the following.

A. Kafka-enabled AAS Repositories as Event Sources

A core component of our architecture involves instrumenting the AAS and Submodel repositories with event-emitting capabilities. We have implemented this functionality and contributed it to the community via the BaSyx reference implementation. This approach allows for granular processing of updates to individual Submodels and even specific Submodel elements as distinct events.

By emitting events for every CRUD (Create, Read, Update, Delete) operation on AAS and Submodel elements, we ensure that changes are captured precisely. These events are published to dedicated Kafka topics, ensuring a reliable and persistent message log. Topic separation by AAS and Submodel and partitioning by AAS ID enables parallel ingestion and message ordering. Fig. 2 illustrates an example of such an event.

B. Template-based Event Processing and Data Ingestion

The events in Kafka emitted from the AAS and Submodel repositories (or the AAS environment) are consumed and

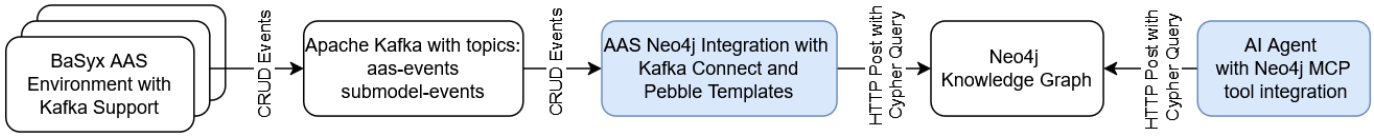

Fig. 1. Overall System Architecture

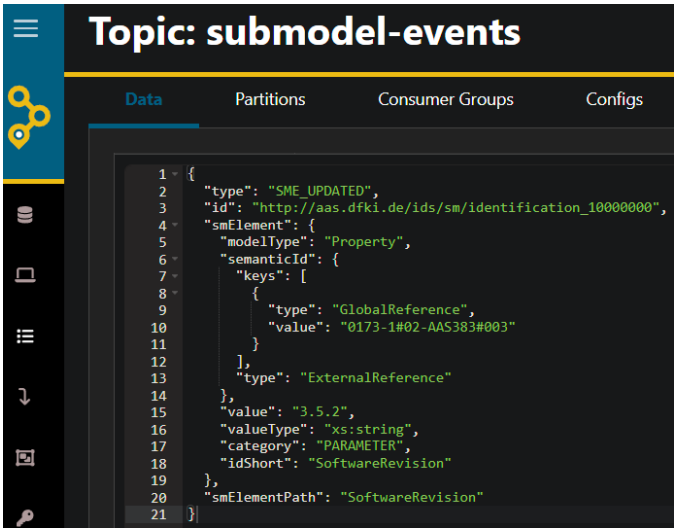

Fig. 2. AKHQ view of a Submodel element update event in Kafka.

processed by Kafka Connect. Kafka Connect serves as a robust and scalable framework for streaming data between Apache Kafka and other systems.

We then use the Pebble Templating Engine [11] for transforming a received Kafka event into a dynamically generated HTTP request body, which encapsulate the Cypher query for updating the Neo4j knowledge graph. It is important to note that these Pebble templates generate YAML, which is then converted to JSON for the HTTP body. This approach is preferred because YAML allows for comments and multiline strings, which are cumbersome or not supported in JSON, thereby simplifying the creation and maintenance of complex templates.

As an example, consider a simplified Pebble template for updating a property within a Submodel:

```
statements:
- statement: |
    MERGE (sme:Property {
      smId: $submodel.id,
      idShortPath: $sme.idShortPath })
    SET sme.value = $sme.value
  parameters:
    submodel:
      id: {{event.id}}
    sme:
      idShortPath: {{event.smElementPath}}
      value: {{event.smElement.value}}
```

This example demonstrates how a received Kafka event (e.g. the one in Fig. 2) containing the Submodel ID, element path, and the new property value, can be directly mapped to a Cypher ‘MERGE‘ and ‘SET‘ statement to update the Neo4j knowledge graph.

To further optimize the flexibility and maintainability of these templates, the underlying model classes represent the POJOs (Plain Old Java Objects) of the Kafka events. These classes are enriched with all necessary data, serving as the context for the Pebble templates. This ensures a consistent and complete data basis for the Cypher queries. Additionally, appropriate indices are automatically created in Neo4j, which are declaratively defined and can be extended, to ensure fast query performance.

In principle, these templates act as customizable blueprints, allowing for no-code adaptation of how to handle the received events. This makes it easy to adapt the templates to a new Neo4j database structure or to provide just another set of templates for a different transformation target, e.g., an OWL-based knowledge graph.

C. Parallel Processing and Event Ordering

A key consideration in this distributed architecture is the independent nature of AAS and Submodel services. Since Submodels can be deployed on different services than shells, their respective CRUD events are written to separate Kafka topics. These distinct event streams can be processed independently and in parallel, enhancing the overall system throughput and scalability.

To ensure data consistency and prevent race conditions (e.g., an update event arriving before its corresponding creation event, or two updates for the same element being processed out of order), the AAS and Submodel event producers utilize the unique identifier of the respective AAS or Submodel as the Kafka message key.

Kafka guarantees strict message ordering per partition for a given key. This means that all events pertaining to a specific AAS or Submodel are delivered to the Kafka Connect consumer tasks in the exact order they were produced.

Kafka Connect, by leveraging its task-based parallelism, can process messages from different partitions (and thus different AAS/Submodels) concurrently, while maintaining the required ordering for each individual AAS or Submodel. This architecture therefore allows for both high throughput through parallel processing and strong consistency guarantees on a per-entity basis.

IV. Representation of AAS Concepts in Neo4j for Validation and Querying

The core of our approach lies in the effective mapping of the hierarchical and relational structure of the AAS metamodel into a graph database model. Neo4j, with its native graph storage and processing capabilities, is ideally suited to represent the complex interconnections within and between AAS instances. This section details the proposed graph schema and the mapping rules applied to AAS metamodel concepts.

A. Mapping Core AAS Concepts to Graph Elements

At the highest level, each AAS and each Submodel is represented as a distinct node in the Neo4j graph.

- **AAS:** An AAS instance is mapped to a node with the label `(:AssetAdministrationShell)`. Its unique identifier (e.g., `id`) and other top-level properties are stored as properties on this node.
- **Submodels:** Each Submodel within an AAS is represented as a node with the label `(:Submodel)`. Similar to AAS nodes, its unique identifier and properties are stored on the node. A direct relationship, `[:HAS_SUBMODEL]`, connects the `(:AssetAdministrationShell)` node to its `(:Submodel)` nodes.
- **Submodel Elements:** The various types of Submodel Elements (e.g., Properties, Files, Collections, Entity, ReferenceElement) are also mapped to individual nodes. Each element receives a specific label corresponding to its type (e.g., `(:Property)`, `(:File)`, `(:Entity)`). Furthermore, leveraging the inheritance hierarchy defined by the IDTA's OpenAPI specification, each Submodel element also receives the `(:SubmodelElement)` label. Their respective properties (e.g., `value`, `idShort`) are stored as node properties. Hierarchical relationships within Submodels (e.g., a `Property` within a `SubmodelElementCollection`) are represented by relationships like `[:HAS_ELEMENT]`. The full `idShort` path is used to uniquely identify elements within a Submodel.

B. Modeling References for Enhanced Semantics and Validation

A critical aspect of AAS is the extensive use of references, which link different parts of an AAS, other AAS, or external concepts. Our graph model specifically emphasizes these references to enable powerful semantic queries and structural validation.

- **Semantic IDs (`semanticId`):** As discussed, `semanticId`s are crucial for conveying the meaning of an AAS element by linking it to standardized concepts (e.g., from ECLASS). We model each unique `semanticId` (e.g., the International Registration Data Identifier (IRDI) "0173-1#2302-AAO677#23002", which describes the manufacturer name) as a distinct `(:SemanticConcept)` node. AAS elements that possess a `semanticId` are connected to the corresponding concept node via a `[:HAS_SEMANTIC]` relationship. This allows for rich semantic queries.
- **Global Asset IDs (`globalAssetId`):** A distinct `(:Asset)` node is created when the Asset Administration Shell (AAS) itself is created. This node is then referenced by the AAS via a `[:HAS_ASSET]` relationship. The `(:Asset)` node stores the `globalAssetId` and other asset-specific properties derived from the AAS's `AssetInformation`. For `Entity` Submodel Elements that reference a global asset, the `(:Entity)` node is connected to this `(:Asset)` node via a `[:HAS_RELATION]` relationship.
- **Other References (e.g., `ReferenceElement`, `RelationshipElement`):** Other types of internal and external references within the AAS are also modeled as relationships between nodes. For instance, a `(:ReferenceElement)` node might point to another AAS element node via a `[:HAS_REFERENCE]` relationship. This `[:HAS_REFERENCE]` relationship carries a property that stores the specific property name from the IDTA service description (e.g., `value` for a `ReferenceElement` or `first` / `second` for a `RelationshipElement`), ensuring adherence to the standard.

C. Graph Schema and Indexing

The resulting Neo4j graph schema consists of various node labels (e.g., `AssetAdministrationShell`, `Submodel`, `Property`, `Entity`, `SemanticConcept`, `Asset`) and relationship types (e.g., `HAS_SUBMODEL`, `HAS_ELEMENT`, `HAS_SEMANTIC`, `HAS_REFERENCE`). To ensure efficient query performance, appropriate indices are created on frequently accessed node properties, such as the `id` of AAS and Submodel nodes and the `idShort` path of Submodel Elements. This comprehensive graph model transforms the AAS data from a hierarchical document structure into an interconnected knowledge graph, enabling powerful traversal and pattern matching capabilities.

D. Structural Validation through Graph Traversal

One of the primary benefits of modeling AAS data as a graph is the ability to perform efficient structural validation. By traversing relationships, we can identify inconsistencies, missing information, or "dangling" references that would be difficult to detect in a document-oriented database.

- **Identifying Dangling References:** A common issue in distributed systems are references pointing to non-existent or invalid targets. In our model, references that point to a target without a `sourceUrl` are considered "dangling references." Only during the actual creation event are referable nodes (AssetAdministrationShell, Asset, Submodel, SubmodelElement) provided with a `sourceUrl`. The `sourceUrl` always refers to the address under which the resource can be queried from the AAS repository. When a reference is created, an object without a

`sourceUrl` is always created using `MERGE` to establish the reference relationship.

```
MATCH (s)-[r:HAS_REFERENCE]->(t)
WHERE t.sourceUrl IS NULL
RETURN s.sourceUrl AS sourceUrl,
  r.type AS relationshipType,
  t.id AS targetId,
  t.id AS path
```

- **Counting Child Elements with 'derivedFrom' Relationship to a Specific AAS:** This query helps to understand the inheritance structure and how many elements are derived from a particular Asset Administration Shell.

```
MATCH (:AssetAdministrationShell)
  -[:HAS_REFERENCE {type:
    'derivedFrom'}]->
     (t:AssetAdministrationShell)
RETURN t.id AS targetId,
  COUNT(*) AS count
ORDER BY count DESC
```

- **Identifying 'derivedFrom' References to Non-Type Assets:** This query identifies shells that are referenced via a 'derivedFrom' relationship but are not of 'Type' assetKind, or where the assetKind is not specified. This helps to find potential inconsistencies in the asset modeling.

```
MATCH (:AssetAdministrationShell)-
  [:HAS_REFERENCE {type: 'derivedFrom'
  }]->(t:AssetAdministrationShell)
  -[:HAS_ASSET]->(a:Asset)
WHERE a.assetKind IS NULL
  OR NOT a.assetKind = 'Type'
RETURN t.id AS targetId,
  a.assetType AS targetAssetType
```

E. Complex Querying for Operational Insights

Beyond validation, the graph model enables complex, multi-hop queries that were previously unfeasible. These queries provide deep operational insights crucial for I4.0 applications like predictive maintenance, asset management, and process optimization.

Let's imagine the following small but in detail complex scenario: a maintenance technician is required to identify all MiR100 autonomous mobile robots from *Mobile Industrial Robots* with software versions older than `2.4.2` in order to perform a software update.

The corresponding AAS of such a MiR100 comprises an *Identification* Submodel containing 21 SubmodelElements (including the software version) and a *Signal* Submodel with 47 SubmodelElements (just as additional payload). Fig. 3 shows the AAS Web UI, with excerpts of the Identification and Signal Submodels.

Then the query can be efficiently executed using Cypher where the string-based software version in the Submodel is split in parts reflecting semantic version to avoid issues in lexical ordering (e.g., 2.18.0 < 2.4.2 in lexical ordering)

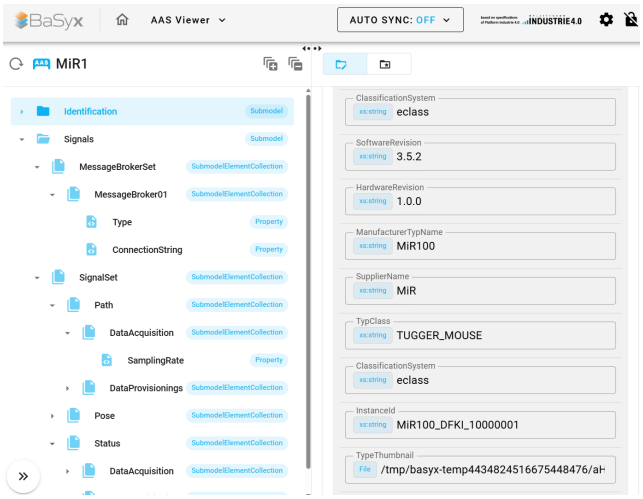

Fig. 3. The AAS Web UI showing the Identification and Signal Submodels

```
MATCH (a:AssetAdministrationShell)-
  [:HAS_SUBMODEL]->(s:Submodel)
MATCH (s)-[:HAS_SEMANTIC_ID]->
  (:SemanticConcept {id:
  'https://www.hsu-hh.de/aut/aas/
    identification'})
MATCH (s)-[:HAS_ELEMENT]->(pMf)-
  [:HAS_SEMANTIC_ID]->(:SemanticConcept
  {id: '0173-1#02-AAO677#002'})
MATCH (s)-[:HAS_ELEMENT]->(pPd)-
  [:HAS_SEMANTIC_ID]->(:SemanticConcept
  {id: '0173-1#02-AAW338#001'})
MATCH (s)-[:HAS_ELEMENT]->(pVers)-
  [:HAS_SEMANTIC_ID]->(:SemanticConcept
  {id: '0173-1#02-AAW338#021'})
WHERE pPd.value = 'MiR100'
WITH a, s, pMf, pPd, pVers,
  split(pVers.value, ".")
  AS versionParts
WITH a, s, pMf, pPd, pVers,
  toInteger(versionParts[0]) AS major,
  toInteger(versionParts[1]) AS minor,
  toInteger(versionParts[2]) AS patch
WHERE
  (major < 2) OR
  (major = 2 AND minor < 4) OR
  (major = 2 AND minor = 4 AND patch < 2)
RETURN a, s, pMf, pPd, pVers
```

This query demonstrates how we can combine information from different parts of the AAS structure (AAS properties, Submodel elements, and semantic references) to answer a complex queries. The results of such a query, as visualized in the Neo4j Browser, are shown in Fig. 4.

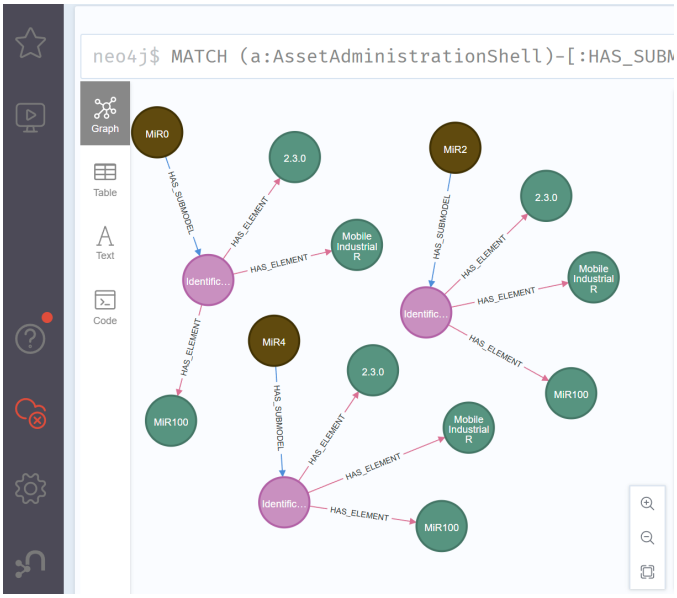

Fig. 4. Graph visualization of AAS instances with software version $\leq$ 2.4.2

V. Evaluation

The presented system was implemented using open-source technologies and deployed in a fully containerized environment with Docker Compose, orchestrating all software components, as depicted in Fig. 5: Apache Kafka (version 7.9.1), Kafka Connect (version 7.9.1), Neo4j (version 5.23.0), the AAS services from eclipsebasyx (version 2.0.0-SNAPSHOT-d81b59c), and the aas-neo4j-kafka container.

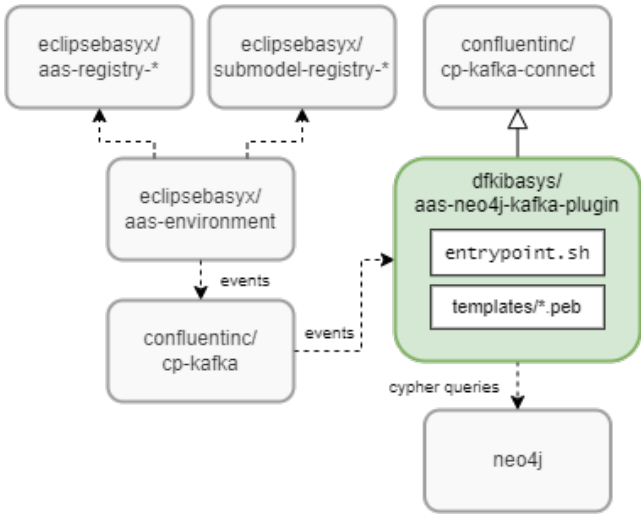

Fig. 5. Docker image and container dependencies

A. Setup

The use case and the Cypher query presented in the last section IV-E form the basis for our evaluation of query performance and expressiveness.

We deployed two different sets of 100 and 1000 shells representing individual MiR100 robots and performed measurements to evaluate both ingestion and query performance under various workloads. The tests were conducted on a Windows PC equipped with a 12th Gen Intel(R) Core(TM) i7-12800HX processor at 2.00 GHz and 64 GB of RAM.

B. Results

Deployment latency: Table I and Table II summarize end-to-end processing times (average per event) for different scales. Even for large batch inserts, most time is spent on the Neo4j transaction (not in the event transformation or network).

Events	Pebble (ms)	Cypher (ms)	Total (ms)
signals-sm	3.60	76.90	81.00
identification-sm	1.10	32.10	33.80
aas-events	0.60	12.80	13.90

TABLE I
AVERAGE PROCESSING TIME FOR 10 SHELLS (30 EVENTS).

Events	Pebble (ms)	Cypher (ms)	Total (ms)
signals-sm	3.55	1293.07	1297.12
identification-sm	1.54	83.75	85.62
aas-events	0.42	141.46	142.42

TABLE II
AVERAGE PROCESSING TIME FOR 100 SHELLS (300 EVENTS).

Increasing Kafka partitions (to 20) and using 10 Kafka Connect tasks did not improve throughput, as Neo4j transaction commit times dominated and remained the bottleneck.

Property update performance: Updating a single property value is very fast (see Table III). Even for 1000 updates, the average latency remains < 6 ms.

N	Transformation (ms)	Cypher (ms)	Total (ms)
100	0.00	4.65	4.76
1000	0.01	5.70	5.99

TABLE III
AVERAGE PROCESSING TIME PER PROPERTY UPDATE EVENT.

Query performance: We evaluated three approaches after deploying 1000 AAS:

- Cypher with version parsing/comparison: 67 ms
- Cypher with simple value lookup: 3 ms
- AAS REST API: 27 ms

Complex logic in Cypher, such as parsing and comparing version strings, leads to increased query times compared to simple lookups. However, Cypher enables expressive, relational queries — such as traversing location relationships — that are impractical using the AAS REST API alone.

It is important to note that this setup, being on a local PC, benefits the traditional AAS REST API approach due to negligible network latency.

C. Discussion

Scalability: Transactional writes in Neo4j are the bottleneck for bulk ingestion; more partitions/tasks in Kafka Connect do

not significantly improve this. Property updates are always fast.

Query Expressiveness: Cypher enables navigation of deep relationships and detection of dangling references, which are not possible or very difficult with AAS APIs. However, computationally heavy queries (e.g., string processing) in Cypher can increase latency.

Practicality: Reproducing such setups (many shells, dangling references, etc.) is complex in classical AAS tooling, while the graph model in Neo4j enables such scenarios and advanced integrity checks directly.

Reliability vs. Performance. Enabling Kafka event streaming for AAS updates results in approximately 33% higher latency for data ingestion, since the system waits for confirmation of event delivery before returning a success status to clients. This synchronous approach was deliberately chosen to prevent scenarios in which an update appears successful at the client, but the corresponding event was not actually delivered to Kafka. While asynchronous event emission could improve throughput, it would compromise delivery guarantees.

Throughput and backpressure. A further consideration for production use is that, in scenarios where updates arrive faster than they can be ingested into Neo4j (e.g., due to high Cypher transaction latencies), the Kafka topics may begin to fill up as events accumulate. While this architecture ensures that no data is lost—even under high load—there is a risk that data synchronization will lag behind the latest state. This effect is especially relevant when bulk deployments or bursts of updates are performed. In practice, however, the majority of real-world workloads are dominated by read access or isolated property updates, which are processed efficiently.

In scenarios where read access to the AAS is dominant or updates only affect individual values, the increased latency during initial resource deployments is usually negligible. Such full deployments mainly occur during system startup.

VI. Use case: LLM Agent with Neo4j MCP server integration

A. Agent Architecture

The Model Context Protocol (MCP) is an open-source standard first published by Anthropic [12] in November 2024 to enable seamless and reusable integration between LLMs and external tools, services, and data sources. MCP has gained much attention since then, and Neo4j provides respective tool integrations in terms of so-called MCP servers to its graph database.

The agentic system was implemented from scratch in Python 3.12. It is composed of an agent driven by Anthropic's reasoning model Claude Sonnet 4 [13], the Neo4j Cypher MCP Server [14], and basic conversational memory. Sonnet 4 was chosen given the good benchmark results it achieves in tool calling, coding, and agentic behavior [15]. The agent runs in a continuous loop until it decides that it has solved the query successfully by using the given MCP tools and its own preexistent abilities. The memory provides coherence between current and previous questions and answers allowing for a more complex interaction with the user, e.g. multi-step analysis of the graph data. The user interface is a simple command line interface (CLI) that allows the user to ask questions, see the answer to the question, and see the intermediate reasoning steps that the agent took to solve the question. Besides using the MCP tools to ask questions about the graph, the agent is also able to convert natural language expressions to Cypher queries and vice-versa. The agent has demonstrated that it is resilient to some errors that arise when using the MCP tools. And it is able to rely on its own knowledge to continue solving the question or to switch to a less obvious or direct tool usage to solve the question. Fig. 6 shows a sequence diagram of the agentic system and its principle workflow.

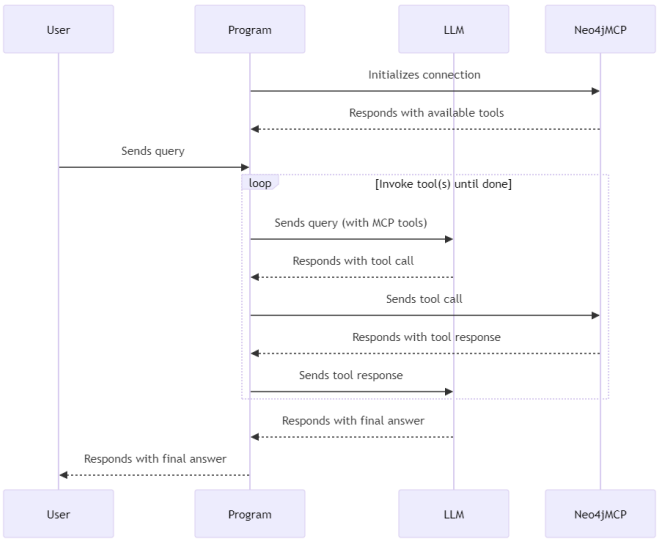

Fig. 6. Sequence diagram of the agentic system

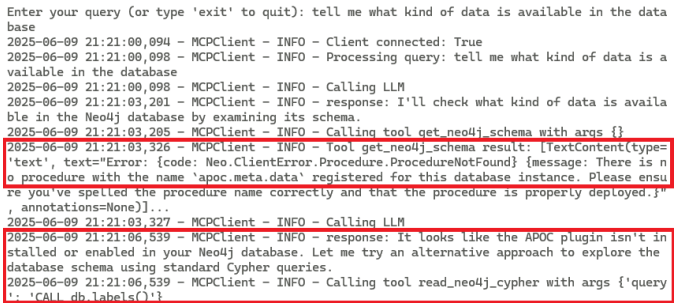

Fig. 7. Agent "recovering" from error and proposing an alternative solution

B. Sample Interactions and Error Recovery

First, we asked the agent to simply explore the available data in the knowledge graph. As shown in Fig. 7, unfortunately, the agent immediately runs into an error due to the absence of a plugin in the Neo4j installation. The agent then decides to solve the query via an alternative way.

After a number of intermediate tool and LLM calls the agent produces an answer to the initial query, see Fig. 8.

```
2025-06-09 21:21:55,645 - MCPClient - INFO - response: Based on my exploration of the databas
e, here's what kind of data is available:

## Database Overview

This Neo4j database contains **Asset Administration Shell (AAS)** data, which is a standardiz
ed digital representation of assets in Industry 4.0. The database follows the AAS metamodel s
tructure.

### Node Types (Labels):
- **AssetAdministrationShell** (6 instances) - The main digital twin containers
- **Asset** (6 instances) - Physical or logical assets being represented
- **Submodel** (12 instances) - Structured data containers within AAS
- **SubmodelElement** types including:
  - **Property** (246 instances) - Key-value data elements
  - **SubmodelElementCollection** (108 instances) - Grouped elements
  - **File** (30 instances) - File references
  - **ReferenceElement** (18 instances) - References to other elements
- **SemanticConcept** (36 instances) - Semantic definitions and concepts

### Relationship Types:
- **HAS_SUBMODEL** - Links AAS to its submodels
- **HAS_ASSET** - Links AAS to assets
- **HAS_ELEMENT** - Links submodels to their elements
- **HAS_REFERENCE** - Reference relationships
- **HAS_SEMANTIC** - Semantic concept relationships

### Data Structure:
The database appears to contain data for **MiR100** robots (mobile industrial robots) with id
entifiers like "mir100_10000000_aas", "mir100_10000001_aas", etc.

Each Asset Administration Shell contains:
- **Identification** submodel (with ~20 elements each)
- **Signals** submodel (with ~2 elements each)

This is a structured industrial IoT/Industry 4.0 dataset that models digital twins of robotic
 assets with their properties, capabilities, and operational data organized according to the
Asset Administration Shell standard.
```

Fig. 8. Final output

```
2025-06-09 19:37:36,186 - MCPClient - INFO - response: Based on my analysis of the Neo4j data
base, here are all the **MiR100 robots that have software versions older than 2.4.2**:

| Robot ID | Software Version | Submodel ID |
|----------|------------------|-------------|
| MiR100_DFKI_10000000 | 2.3.0 | http://aas.dfki.de/ids/sm/identification_10000000 |
| MiR100_DFKI_10000002 | 2.3.0 | http://aas.dfki.de/ids/sm/identification_10000002 |
| MiR100_DFKI_10000004 | 2.3.0 | http://aas.dfki.de/ids/sm/identification_10000004 |

**Summary:**
- **3 MiR100 robots** have software versions older than 2.4.2
- All of them are running **version 2.3.0**
- The other MiR100 robots in the database (MiR100_DFKI_10000001, MiR100_DFKI_10000003, and Mi
R100_DFKI_10000005) have newer versions (2.7.0 and 2.18.0) that are above the 2.4.2 threshold

These robots may need software updates if version 2.4.2 or newer is required for your operati
ons.
```

Fig. 9. MiR100 robots that have a software version older than 2.4.2.

Then, we asked the agent *"Please show me all MiR100 autonomous mobile robots that have a software version older than 2.4.2."*. The agent remembers the previous error and avoids it this time. Again, after a number of intermediate tool and LLM calls the agent correctly produces the output shown in Fig. 9.

VII. Conclusion and Future Work

This paper presented an event-driven architecture for bootstrapping and maintaining a synchronized Neo4j knowledge graph based on the BaSyx AAS Environment and Apache Kafka. We demonstrated how this approach enables powerful, semantically rich Cypher queries and significantly improves structural validation capabilities. Our model effectively leverages the hierarchical and relational structures of AAS elements, ensuring a robust and continuously updated knowledge graph. Our evaluation highlights that write operations to Neo4j, especially during bulk deployment, are comparatively costly. However, read access and complex graph queries remain highly efficient and are a major advantage over current AAS repository implementations, as only the graph-based approach enables practical, expressive navigation across deeply linked industrial data. This paves the way for the development of conversational AI agents, making the complex semantic depth of AAS data more intuitively accessible and actionable for industrial users.

We received already promising results out-of-the-box with Claude Sonnet 4 as a commercial online reasoning model. In the future, we expect the development of AAS-centric MCP tools and the rise of open small reasoning models that will be fine-tuned to AAS specifics in order to allow for local inference directly on-premise.

The source code of the AAS Neo4j Integration [16] with Docker setup and example workloads as well as the AI agent [17] are available for transparency, reproducibility, and dissemination at GitHub.

Acknowledgment

This work has been funded by the German Ministry for Education and Research (BMBF) in the context of the project BaSys4Transfer (01IS22089C).

References

[1] Industrial Digital Twin Association, "Specification of the Asset Administration Shell Part 1: Metamodel – IDTA Number: 01001," https://industrialdigitaltwin.io/aas-specifications/IDTA-01001/v3.1/index.html, 2025.

[2] Eclipse Basyx, "AAS Web UI," https://wiki.basyx.org/en/latest/content/user_documentation/basyx_components/web_ui/index.html, 2025.

[3] Industrial Digital Twin Association, "Specification of the Asset Administration Shell Part 2: Application Programming Interfaces – IDTA Number: 01002," https://industrialdigitaltwin.io/aas-specifications/IDTA-01002/v3.1/index.html, 2025.

[4] Neo4J, "Cypher Graph Query Language," https://neo4j.com/product/cypher-graph-query-language/, 2025.

[5] I. Grangel-González, L. Halilaj, G. Coşkun, S. Auer, D. Collarana, and M. Hoffmeister, "Towards a semantic administrative shell for industry 4.0 components," *2016 IEEE Tenth International Conference on Semantic Computing (ICSC)*, pp. 230–237, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:18254287

[6] A. Bozkurt and R. Schulz, "Asset administration shell and knowledge graph-based production logistics planning in fluid manufacturing systems," *Logistics Journal: referierte Veröffentlichungen*, vol. 2023, no. 1, 2023.

[7] M. H. Rimaz, C. Plociennik, and M. Ruskowski, "Semantic asset administration shell for circular economy," in *KG4S@ ESWC*, 2024.

[8] P. Rübel, N. Moarefvand, W. Motsch, A. Wagner, and M. Ruskowski, "Enabling fault diagnosis in skill-based production environments," in *2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2023, pp. 1–8.

[9] T. Schmeyer, K. Krämer, A.-L. Peh, B. Brandherm, M. Chikobava, and G.-L. Kiefer, "Digital twin data broker with assisted mapping into a knowledge base," in *Innovative Intelligent Industrial Production and Logistics*, M. Dassisti, K. Madani, and H. Panetto, Eds. Cham: Springer Nature Switzerland, 2025, pp. 23–40.

[10] A. Luxenburger, D. Porta, S. Knoch, J. Mohr, and T. Schwartz, "A service infrastructure for industrie 4.0 testbeds based on asset administration shells," in *2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2023, pp. 1–8.

[11] PebbleTemplates, "Pebble Java Templating Engine," https://pebbletemplates.io, 2025.

[12] Anthropic, "Model Context Protocol," https://www.anthropic.com/news/model-context-protocol, 2024.

[13] ——, "Claude Sonnet 4," https://docs.anthropic.com/en/docs/about-claude/models/overview#model-comparison-table, 2025.

[14] Neo4J, "Cypher MCP Server," https://github.com/neo4j-contrib/mcp-neo4j/tree/main/servers/mcp-neo4j-cypher, 2025.

[15] LiveBench, "Leaderboard," https://livebench.ai//?Agentic+Coding=as#/, 2025.

[16] Gerhard Sonnenberg, "AAS Neo4j Integration," https://github.com/dfkibasys/aas_neo4j_integration, 2025.

[17] Fabio Espinosa, "AAS Neo4j Agent with MCP," https://github.com/dfkibasys/aas_neo4j_agent, 2025.